

executed_plan

August 23, 2026

1 AML Alert Triage — Plan to Achieve an Additional 5% Discard

An executable implementation of the experimental plan for moving from ~5% additional discard toward 10%+ **while preserving STR / important-case capture**.

Framing. The objective is *not* “hit 10%”. It is to establish the **discard-vs-recall frontier**, then let the MLRO choose an operating point on it with a pre-approved loss budget. Production remains rule-based; ML is used offline only, to discover auditable scorecard and SQL logic.

1.1 Experiment matrix

#	Experiment	Question	Priority
1	Translation loss	Is SQL losing model discard capacity?	1
2	Scorecard	Can additive logic recover interactions?	2
3	Residual AUC	Is there signal in remaining alerts?	3
4	Counterparty / network	Does breadth add new signal?	4
5	Peer deviation	Are customers unusual vs <i>comparable</i> customers?	5
6	Timing / rhythm	Does temporal behavior separate low-risk alerts?	6
7	Cross-alert history	Does prior alert behavior add safe signal?	7
8	Three-tier queue	Can workload fall without forcing permanent discard?	8

1.2 Notebook structure

- §1 Setup and synthetic residual population (replace with a real extract)

- §2 Freeze the current baseline (the 5% reference point)
- §3 Experiment 3 — residual AUC diagnostic (*run this early; it decides the branch*)
- §4 Experiments 4-7 — new feature families, tested incrementally
- §5 Experiment 1 — translation loss (model capacity vs. deployed SQL)
- §6 Experiment 2 — SHAP scorecard, and why AND-chains under-discard
- §7 The discard-vs-recall frontier
- §8 Experiment 8 — three-tier operating design
- §9 Decision sequence and sprint plan

Dependencies: pandas numpy scikit-learn xgboost shap matplotlib

```
[1]: import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.metrics import roc_auc_score, roc_curve, recall_score,
    ↪ precision_score, fbeta_score

import xgboost as xgb
import shap

RANDOM_STATE = 42
np.random.seed(RANDOM_STATE)
BETA = 2.0                # recall-weighted, consistent with the primary triage
    ↪ model
pd.set_option("display.width", 120)
```

1.3 1. Setup — construct the residual population

1.3.1 1.1 What “residual population” means here

Per the plan (§4, Priority 3), the model must be trained and evaluated **specifically on alerts that survive the current suppression and triage layers** — not on all alerts.

ALL ALERTS

Mandatory / policy rules	→ INVESTIGATE	(excluded from this analysis)
Whitelist	→ suppressed	(excluded)
Current discard rules (~5%)	→ discarded	(the frozen baseline, §2)
Everything else	→ RESIDUAL POPULATION	← this notebook

Measuring on the full alert population would flatter the results, because the easy suppressions have already been taken by upstream layers.

1.3.2 1.2 Synthetic data

The generator below produces a residual population with a realistic ~6% important rate and includes **all four new feature families** from the plan so each can be tested incrementally in §4. Replace `load_residual_population()` with a real extract.

```

[2]: def load_residual_population(n=12000, seed=RANDOM_STATE):
    """
    STUB - replace with a real extract of alerts that survive existing layers,
    ↪e.g.:

        SELECT a.alert_id, a.customer_id, a.alert_date, a.scenario_id,
               f.*, d.outcome
        FROM alerts a
        JOIN customer_features f ON ... AND f.as_of_date = a.alert_date
        JOIN dispositions d     ON a.alert_id = d.alert_id
        WHERE a.alert_id NOT IN (SELECT alert_id FROM mandatory_routed)
        AND a.customer_id NOT IN (SELECT customer_id FROM whitelist_asof(a.
    ↪alert_date))
        AND d.case_status = 'Investigated'

    Returns one row per alert with baseline + new-family features and a binary
    ↪label.
    """
    rng = np.random.default_rng(seed)

    # ---- latent risk drives the label and correlates with several features
    ↪-----
    latent = rng.beta(a=2.0, b=28, size=n)

    # ---- segment / peer-group structure
    ↪-----
    segment = rng.choice(["retail", "sme", "corporate"], size=n, p=[0.62, 0.28,
    ↪0.10])
    seg_scale = pd.Series(segment).map({"retail": 1.0, "sme": 3.5, "corporate":
    ↪12.0}).to_numpy()
    scenario_id = rng.choice([f"S{i}" for i in range(1, 13)], size=n)

    # ---- BASELINE family: amount / velocity profiles
    ↪-----
    account_age_months = np.clip(rng.exponential(36, n), 1, 240)
    has_full_6m = (account_age_months >= 6).astype(int)

    avg_amt_6m_true = rng.lognormal(mean=7.6 + np.log(seg_scale), sigma=0.9,
    ↪size=n)
    avg_amt_2m = avg_amt_6m_true * (1 + latent * rng.normal(1.4, 0.5, n).
    ↪clip(0))
    avg_amt_6m = np.where(has_full_6m, avg_amt_6m_true, np.nan)
    std_amt_6m = np.where(has_full_6m, avg_amt_6m_true * rng.uniform(0.1, 0.6,
    ↪n) * (1 + latent), np.nan)

```

```

    txn_count_2m = rng.poisson(lam=(4 + latent * 35) * np.sqrt(seg_scale),
↪size=n).astype(float)
    txn_count_6m_true = rng.poisson(lam=(4 + latent * 30) * np.sqrt(seg_scale),
↪* 3, size=n).astype(float)
    txn_count_6m = np.where(has_full_6m, txn_count_6m_true, np.nan)

    # ---- FAMILY 4: counterparty / network breadth
↪-----
    distinct_cp_6m = rng.poisson(lam=2 + latent * 28 + seg_scale * 0.4, size=n).
↪astype(float)
    new_cp_6m = rng.binomial(np.maximum(distinct_cp_6m, 1).astype(int),
                             np.clip(0.15 + latent * 1.6, 0, 0.95))
    new_cp_ratio = new_cp_6m / np.maximum(distinct_cp_6m, 1)
    inbound_cp_6m = rng.binomial(np.maximum(distinct_cp_6m, 1).astype(int), 0.5)
    outbound_cp_6m = np.maximum(distinct_cp_6m - inbound_cp_6m, 0)
    # concentration: share of volume through the single largest counterparty
    largest_cp_share = np.clip(rng.beta(2, 3 + latent * 6, n), 0.01, 0.99)
    days_active_6m = np.clip(rng.poisson(lam=12 + latent * 55, size=n), 1, 180).
↪astype(float)
    cp_per_active_day = distinct_cp_6m / days_active_6m

    # ---- FAMILY 6: timing / rhythm
↪-----
    txn_per_active_day = txn_count_2m / np.maximum(days_active_6m / 3, 1)
    active_day_ratio = days_active_6m / 180.0
    max_txn_per_day = rng.poisson(lam=1 + latent * 14, size=n).astype(float)
    median_txn_per_day = np.maximum(txn_per_active_day * rng.uniform(0.3, 0.9,
↪n), 0)
    # burstiness: dispersion of inter-transaction gaps (higher = spikier)
    burstiness = np.clip(rng.normal(0.3 + latent * 1.5, 0.25, n), 0, 3)
    night_ratio = np.clip(rng.beta(2, 8, n) + latent * 0.5, 0, 1)
    weekend_ratio = np.clip(rng.beta(2, 7, n) + latent * 0.3, 0, 1)

    # ---- FAMILY 7: cross-alert history (leakage-safe by construction)
↪-----
    prior_alert_count = rng.poisson(lam=0.6 + latent * 4.5, size=n).
↪astype(float)
    prior_fp_count = rng.binomial(prior_alert_count.astype(int), 0.85)
    prior_important_count = np.maximum(prior_alert_count - prior_fp_count, 0)
    days_since_last_alert = rng.exponential(scale=np.maximum(120 - latent * 90,
↪15), size=n)
    same_scenario_alert_count = rng.poisson(lam=0.3 + latent * 2.2, size=n).
↪astype(float)

    df = pd.DataFrame({
        "alert_id": [f"A{i:07d}" for i in range(n)],

```

```

"customer_id": [f"C{i:07d}" for i in range(n)],
"scenario_id": scenario_id,
"segment": segment,
# baseline
"avg_amt_2m": avg_amt_2m,
"avg_amt_6m": avg_amt_6m,
"ratio_amt_2v6m": avg_amt_2m / avg_amt_6m,
"std_amt_6m": std_amt_6m,
"txn_count_2m": txn_count_2m,
"txn_count_6m": txn_count_6m,
"ratio_txn_2v6m": txn_count_2m / txn_count_6m,
"account_age_months": account_age_months,
"has_full_6m_history": has_full_6m,
# counterparty / network
"distinct_cp_6m": distinct_cp_6m,
"new_cp_6m": new_cp_6m,
"new_cp_ratio": new_cp_ratio,
"inbound_cp_6m": inbound_cp_6m,
"outbound_cp_6m": outbound_cp_6m,
"largest_cp_share": largest_cp_share,
"cp_per_active_day": cp_per_active_day,
# timing / rhythm
"txn_per_active_day": txn_per_active_day,
"active_day_ratio": active_day_ratio,
"max_txn_per_day": max_txn_per_day,
"median_txn_per_day": median_txn_per_day,
"burstiness": burstiness,
"night_ratio": night_ratio,
"weekend_ratio": weekend_ratio,
# cross-alert history
"prior_alert_count": prior_alert_count,
"prior_fp_count": prior_fp_count,
"prior_important_count": prior_important_count,
"days_since_last_alert": days_since_last_alert,
"same_scenario_alert_count": same_scenario_alert_count,
})

```

```

# ---- label_

```

```

logit = (
    -4.5
    + 9.5 * latent
    + 1.5 * new_cp_ratio
    + 0.9 * burstiness
    + 0.5 * np.log1p(prior_important_count)
    + 0.25 * np.log1p(distinct_cp_6m)
    - 0.10 * np.log1p(account_age_months)
)

```

```

    + rng.normal(0, 0.35, n)
)
df["is_important"] = (rng.random(n) < 1 / (1 + np.exp(-logit))).astype(int)

# temporal ordering for the temporal split (S validation requirement)
df["alert_date"] = pd.Timestamp("2024-01-01") + pd.to_timedelta(
    rng.integers(0, 540, n), unit="D"
)

return df.replace([np.inf, -np.inf], np.nan).sort_values("alert_date").
↪reset_index(drop=True)

residual = load_residual_population()
print(f"Residual population: {len(residual):,} alerts")
print(f"Important rate:      {residual['is_important'].mean():.2%} "
      f"({residual['is_important'].sum():,} positives)")
print(f>Date range:        {residual['alert_date'].min().date()} to ↪
↪{residual['alert_date'].max().date()})")
residual.head(3)

```

```

Residual population: 12,000 alerts
Important rate:      6.44% (773 positives)
Date range:        2024-01-01 to 2025-06-23

```

```

[2]:  alert_id customer_id scenario_id segment  avg_amt_2m  avg_amt_6m
ratio_amt_2v6m  std_amt_6m  txn_count_2m  \
0  A0001362    C0001362          S4  retail    854.799441    829.358402
1.030676  339.451909          9.0
1  A0007450    C0007450          S12  retail    2060.854350    1777.817673
1.159205  849.608890          5.0
2  A0003938    C0003938          S7  retail    1589.358984    1293.273278
1.228943  287.145360          4.0

    txn_count_6m  ...  burstiness  night_ratio  weekend_ratio  prior_alert_count
prior_fp_count  \
0          17.0  ...    0.175983    0.188162    0.180234                0.0
0
1          16.0  ...    0.349623    0.290524    0.342108                0.0
0
2          28.0  ...    0.848987    0.237854    0.206559                4.0
3

    prior_important_count  days_since_last_alert  same_scenario_alert_count
is_important  alert_date
0                0.0                38.821234                0.0
0  2024-01-01

```

1	0.0	32.613428	1.0
0 2024-01-01			
2	1.0	254.127002	0.0
0 2024-01-01			

[3 rows x 34 columns]

1.3.3 1.3 Feature families

Defined explicitly so each can be added **incrementally** in §4 — the point is to measure what each new family contributes, not to throw everything in at once.

```
[3]: FAMILIES = {
    "baseline": [
        "avg_amt_2m", "avg_amt_6m", "ratio_amt_2v6m", "std_amt_6m",
        "txn_count_2m", "txn_count_6m", "ratio_txn_2v6m",
        "account_age_months", "has_full_6m_history",
    ],
    "counterparty": [
        "distinct_cp_6m", "new_cp_6m", "new_cp_ratio", "inbound_cp_6m",
        "outbound_cp_6m", "largest_cp_share", "cp_per_active_day",
    ],
    "timing": [
        "txn_per_active_day", "active_day_ratio", "max_txn_per_day",
        "median_txn_per_day", "burstiness", "night_ratio", "weekend_ratio",
    ],
    "cross_alert": [
        "prior_alert_count", "prior_fp_count", "prior_important_count",
        "days_since_last_alert", "same_scenario_alert_count",
    ],
    # peer-deviation features are derived in §4.2 and appended there
    "peer": [],
}

for fam, cols in FAMILIES.items():
    print(f"{fam:14s} {len(cols):2d} features")
```

baseline	9 features
counterparty	7 features
timing	7 features
cross_alert	5 features
peer	0 features

1.3.4 1.4 Temporal split

Per the plan's validation requirement (§10): **temporal train/test split, never random**. Train on the earlier period, test on the later one — this simulates the real deployment condition where the model only ever sees the past.

```
[4]: SPLIT_DATE = residual["alert_date"].quantile(0.70)

train_mask = residual["alert_date"] <= SPLIT_DATE
test_mask = ~train_mask

train_df = residual[train_mask].copy()
test_df = residual[test_mask].copy()

print(f"Split date: {SPLIT_DATE.date()}")
print(f"Train: {len(train_df):,} alerts ({train_df['is_important'].mean():.2%}↳
↳important)")
print(f"Test: {len(test_df):,} alerts ({test_df['is_important'].mean():.2%}↳
↳important)")
```

```
Split date: 2025-01-08
Train: 8,402 alerts (6.24% important)
Test: 3,598 alerts (6.92% important)
```

1.4 2. Freeze the current baseline

The plan’s decision sequence (§13) starts with “**freeze the current 5% baseline**”. Every subsequent experiment is measured as a delta against this reference — otherwise improvements can’t be attributed to any particular change.

Replace `current_sql_discard_rule()` with your actual deployed logic. The synthetic version below is written as an **AND-chain**, which is the structure the plan identifies as the likely cause of the ceiling (see §6.1).

```
[5]: def current_sql_discard_rule(df):
    """
    STUB - replace with your deployed discard logic, transcribed exactly.

    Written here as an AND-chain, matching the typical Form-1 deployment:
    an alert must clear EVERY condition to be discarded.
    """
    return (
        (df["avg_amt_2m"] < df["avg_amt_2m"].quantile(0.55)) &
        (df["txn_count_2m"] < df["txn_count_2m"].quantile(0.60)) &
        (df["distinct_cp_6m"] < df["distinct_cp_6m"].quantile(0.60)) &
        (df["account_age_months"] > 12)
    )

def evaluate_discard(discard_mask, y_true, label="rule"):
    """Discard rate, important-case recall retained, and important cases lost.
    ↳ """
    discard_mask = np.asarray(discard_mask).astype(bool)
    y_true = np.asarray(y_true).astype(int)
```

```

n = len(y_true)
n_important = y_true.sum()
important_discarded = int((discard_mask & (y_true == 1)).sum())

return {
    "label": label,
    "discard_rate": discard_mask.mean(),
    "n_discarded": int(discard_mask.sum()),
    "important_lost": important_discarded,
    "important_recall_retained": 1 - important_discarded / n_important if n_important
    else np.nan,
    "fp_reduction": (discard_mask & (y_true == 0)).sum() / max((y_true == 0).sum(), 1),
}

baseline_mask = current_sql_discard_rule(test_df)
baseline = evaluate_discard(baseline_mask, test_df["is_important"], "Current
SQL (frozen baseline)")

BASELINE_RECALL = baseline["important_recall_retained"]
BASELINE_DISCARD = baseline["discard_rate"]

print(f"FROZEN BASELINE (test period)")
print(f" Discard rate: {baseline['discard_rate']:.2%}")
print(f" Important cases lost: {baseline['important_lost']}")
print(f" Important recall retained: {baseline['important_recall_retained']:.2%}")
print(f" False-positive reduction: {baseline['fp_reduction']:.2%}")
print()
print(f" TARGET: >= {BASELINE_DISCARD + 0.05:.2%} discard at >= {BASELINE_RECALL:.2%} recall retained")

```

FROZEN BASELINE (test period)

```

Discard rate:          17.54%
Important cases lost:   21
Important recall retained: 91.57%
False-positive reduction: 18.21%

```

TARGET: >= 22.54% discard at >= 91.57% recall retained

1.5 3. Experiment 3 — Residual AUC diagnostic

Run this early — it decides which branch of the plan applies.

AUC	Interpretation	Action
0.70	Useful signal exists	Threshold/extraction problem → prioritize §5-6 (translation loss, scorecard)
0.55 – 0.65	Weak signal	Needs genuinely new feature types → prioritize §4
0.50	Little discriminative signal	No technique reaches the target safely → go to §8 (three-tier)

```
[6]: def fit_and_score(feature_cols, train_df, test_df, label="model", verbose=True):
    """Train XGBoost on given features; return AUC, the model, and test_
    ↪ probabilities."""
    X_tr, y_tr = train_df[feature_cols], train_df["is_important"]
    X_te, y_te = test_df[feature_cols], test_df["is_important"]

    spw = (y_tr == 0).sum() / max((y_tr == 1).sum(), 1)

    model = xgb.XGBClassifier(
        n_estimators=300, max_depth=4, learning_rate=0.05,
        subsample=0.9, colsample_bytree=0.9,
        scale_pos_weight=spw,          # class weighting, not SMOTE (plan §10)
        eval_metric="logloss", random_state=RANDOM_STATE, verbosity=0,
    )
    model.fit(X_tr, y_tr)

    proba = model.predict_proba(X_te)[: , 1]
    auc = roc_auc_score(y_te, proba)

    if verbose:
        print(f"{label:34s} features={len(feature_cols):3d}   AUC={auc:.4f}")
        return {"label": label, "auc": auc, "model": model, "proba": proba,
        ↪ "features": feature_cols}

baseline_result = fit_and_score(FAMILIES["baseline"], train_df, test_df,
    ↪ "Baseline features only")

auc = baseline_result["auc"]
if auc >= 0.70:
    verdict = "SIGNAL PRESENT -> threshold/extraction problem. Prioritize_
    ↪ translation loss + scorecard (§5-6).\"
elif auc >= 0.55:
    verdict = "WEAK SIGNAL -> needs new feature types. Prioritize counterparty_
    ↪ + peer deviation (§4).\"
else:
```

```

    verdict = "LITTLE SIGNAL -> target unlikely to be safely reachable. Go to_
↳three-tier design (§8)."

print()
print(f"AUC = {auc:.4f}")
print(verdict)

```

Baseline features only features= 9 AUC=0.6003

AUC = 0.6003

WEAK SIGNAL -> needs new feature types. Prioritize counterparty + peer deviation (§4).

1.6 4. Experiments 4-7 — New feature families

Added **incrementally**, so each family’s marginal contribution is measurable. The plan flags counterparty breadth and peer deviation as the high-priority new information axes — the reasoning being that ratios of amounts to amounts re-express existing information, whereas these represent genuinely different measurement axes.

1.6.1 4.1 Experiment 4 — Counterparty / network breadth

```

[7]: cp_result = fit_and_score(
    FAMILIES["baseline"] + FAMILIES["counterparty"],
    train_df, test_df, "+ Counterparty / network"
)
print(f"    delta vs baseline: {cp_result['auc'] - baseline_result['auc']:+.4f}_
↳AUC")

```

+ Counterparty / network features= 16 AUC=0.6557
 delta vs baseline: +0.0554 AUC

1.6.2 4.2 Experiment 5 — Peer-group deviation

The plan is specific and worth restating: **a peer group means customers with comparable legitimate behavioral/business characteristics, not merely customers that triggered the same rule.** Three definitions are tested:

Definition	Grouping
A — Scenario	same scenario only
B — Behavioral	customer/account segment + transaction-volume band
C — Hybrid	same scenario + segment + volume band

Peer features are **within-peer percentile ranks** — the question being “is this customer unusual relative to *comparable* customers”, which is a different question from “is this customer unusual relative to their own history”.

Leakage note: peer percentiles are computed on the *training* period only and applied to test rows via the frozen bin edges, so no test-period information leaks backward into the reference distribution.

```
[8]: PEER_BASE_FEATURES = [
    "avg_amt_2m", "txn_count_2m", "distinct_cp_6m",
    "inbound_cp_6m", "outbound_cp_6m", "active_day_ratio",
]

def add_volume_band(train_df, test_df, n_bands=4):
    """Volume band from TRAINING quantiles only; applied to both sets."""
    edges = np.array(train_df["txn_count_2m"].quantile(np.linspace(0, 1,
    ↪n_bands + 1)).values, dtype=float, copy=True)
    edges[0], edges[-1] = -np.inf, np.inf
    edges = np.unique(edges)
    for df in (train_df, test_df):
        df["volume_band"] = pd.cut(df["txn_count_2m"], bins=edges,
    ↪labels=False, include_lowest=True)
    return train_df, test_df

def build_peer_features(train_df, test_df, peer_keys, suffix):
    """
    Within-peer percentile rank for each base feature.
    Reference distribution comes from TRAINING rows only (leakage-safe).
    """
    new_cols = []
    for feat in PEER_BASE_FEATURES:
        col = f"peer_pct_{feat}_{suffix}"
        new_cols.append(col)

    # reference values per peer group, from training only
    ref = train_df.groupby(peer_keys, observed=True)[feat].apply(lambda s:
    ↪np.sort(s.dropna().values))

    def pct_rank(row):
        key = tuple(row[k] for k in peer_keys) if len(peer_keys) > 1 else
    ↪row[peer_keys[0]]
        vals = ref.get(key)
        if vals is None or len(vals) == 0 or pd.isna(row[feat]):
            return np.nan
        return np.searchsorted(vals, row[feat], side="right") / len(vals)

    train_df[col] = train_df.apply(pct_rank, axis=1)
    test_df[col] = test_df.apply(pct_rank, axis=1)
```

```

    return train_df, test_df, new_cols

train_df, test_df = add_volume_band(train_df, test_df)

peer_defs = {
    "A_scenario": ["scenario_id"],
    "B_behavioral": ["segment", "volume_band"],
    "C_hybrid": ["scenario_id", "segment", "volume_band"],
}

peer_results = {}
peer_cols_by_def = {}

for name, keys in peer_defs.items():
    train_df, test_df, cols = build_peer_features(train_df, test_df, keys, name)
    peer_cols_by_def[name] = cols
    peer_results[name] = fit_and_score(
        FAMILIES["baseline"] + FAMILIES["counterparty"] + cols,
        train_df, test_df, f"+ Peer deviation ({name})"
    )
    print(f"    delta vs counterparty-only: {peer_results[name]['auc'] -
↪cp_result['auc']:.4f} AUC")

```

```

+ Peer deviation (A_scenario)      features= 22   AUC=0.6478
  delta vs counterparty-only: -0.0079 AUC

+ Peer deviation (B_behavioral)    features= 22   AUC=0.6682
  delta vs counterparty-only: +0.0125 AUC

+ Peer deviation (C_hybrid)        features= 22   AUC=0.6608
  delta vs counterparty-only: +0.0051 AUC

```

```

[9]: best_peer_def = max(peer_results, key=lambda k: peer_results[k]["auc"])
    FAMILIES["peer"] = peer_cols_by_def[best_peer_def]

    print(f"Best peer definition: {best_peer_def} ↵
↪(AUC={peer_results[best_peer_def]['auc']:.4f})")
    print(f"Carrying forward {len(FAMILIES['peer'])} peer features")

```

```

Best peer definition: B_behavioral (AUC=0.6682)
Carrying forward 6 peer features

```

1.6.3 4.3 Experiment 6 — Timing / rhythm

```

[10]: timing_result = fit_and_score(
    FAMILIES["baseline"] + FAMILIES["counterparty"] + FAMILIES["peer"] +
↪FAMILIES["timing"],
    train_df, test_df, "+ Timing / rhythm"

```

```
)
print(f"    delta: {timing_result['auc'] - peer_results[best_peer_def]['auc']:+.
↪4f} AUC")
```

```
+ Timing / rhythm                      features= 29    AUC=0.6835
    delta: +0.0152 AUC
```

1.6.4 4.4 Experiment 7 — Cross-alert history

Leakage warning. These features must only use alert history strictly *before* the current alert's date, and must never encode the current alert's own outcome. `prior_important_count` is legitimate (past dispositions, already known at decision time); anything derived from the present case is not.

```
[11]: full_features = (
        FAMILIES["baseline"] + FAMILIES["counterparty"] + FAMILIES["peer"]
        + FAMILIES["timing"] + FAMILIES["cross_alert"]
    )

full_result = fit_and_score(full_features, train_df, test_df, "+ Cross-alert_
↪history (ALL)")
print(f"    delta: {full_result['auc'] - timing_result['auc']:+.4f} AUC")
```

```
+ Cross-alert history (ALL)            features= 34    AUC=0.6655
    delta: -0.0179 AUC
```

1.6.5 4.5 Incremental contribution summary

```
[12]: increments = [
        baseline_result,
        cp_result,
        peer_results[best_peer_def],
        timing_result,
        full_result,
    ]

inc_df = pd.DataFrame([{"step": r["label"], "n_features": len(r["features"]),
↪ "auc": r["auc"]} for r in increments])
inc_df["delta"] = inc_df["auc"].diff().fillna(0)

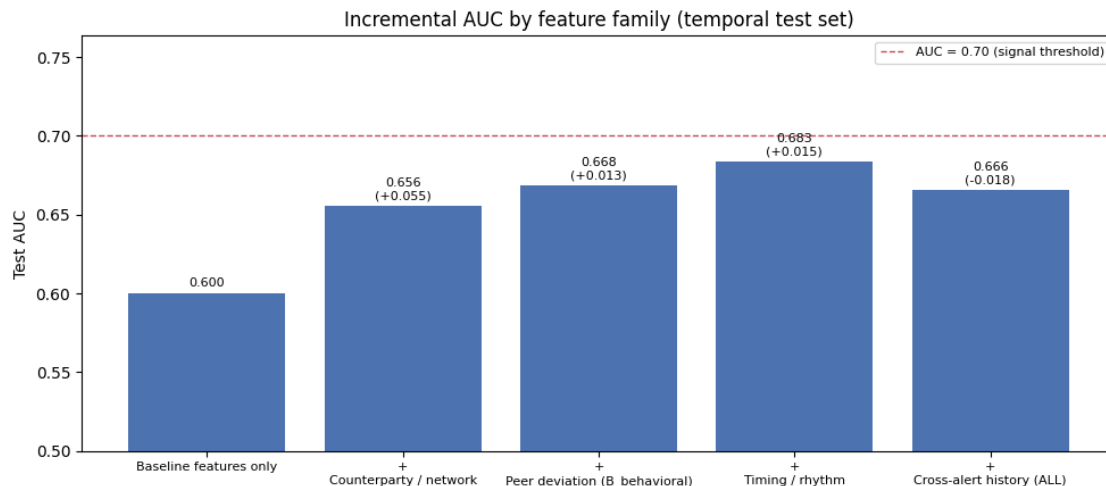
fig, ax = plt.subplots(figsize=(10, 4.5))
bars = ax.bar(range(len(inc_df)), inc_df["auc"], color="#4c72b0")
for i, (a, d) in enumerate(zip(inc_df["auc"], inc_df["delta"])):
    ax.text(i, a + 0.004, f"{a:.3f}" + (f"\n({d:+.3f})" if i else ""),
↪ ha="center", fontsize=8)
ax.axhline(0.70, color="#c44e52", linestyle="--", linewidth=1, label="AUC = 0.
↪70 (signal threshold)")
ax.set_xticks(range(len(inc_df)))
```

```

ax.set_xticklabels([s.replace("+ ", "+\n") for s in inc_df["step"]], fontsize=8)
ax.set_ylabel("Test AUC")
ax.set_ylim(0.5, min(1.0, inc_df["auc"].max() + 0.08))
ax.set_title("Incremental AUC by feature family (temporal test set)")
ax.legend(fontsize=8)
plt.tight_layout()
plt.show()

inc_df.round(4)

```



```

[12]:
step  n_features  auc  delta
0      Baseline features only      9  0.6003  0.0000
1      + Counterparty / network     16  0.6557  0.0554
2  + Peer deviation (B_behavioral)   22  0.6682  0.0125
3      + Timing / rhythm            29  0.6835  0.0152
4      + Cross-alert history (ALL)   34  0.6655 -0.0179

```

1.6.6 4.6 Select the best feature set — more features is not automatically better

The incremental test above exists precisely so that a family which *hurts* generalization can be caught and dropped. On a temporal test set, a family can reduce AUC when it encodes period-specific behavior that does not carry forward — cross-alert history is a common culprit, since alerting patterns shift as rules and investigator practice change over time.

Rather than assume the full stack is best, select the feature set that actually maximized held-out AUC and carry **that** forward into §5-6.

```

[13]: best_step = max(increments, key=lambda r: r["auc"])
      BEST_FEATURES = best_step["features"]
      best_model_result = best_step

```

```

print(f"Best feature set: {best_step['label']}")
print(f"  {len(BEST_FEATURES)} features, test AUC = {best_step['auc']:.4f}")

dropped = [f for f in full_result["features"] if f not in BEST_FEATURES]
if dropped:
    print(f"\nDropped {len(dropped)} features that did not improve held-out AUC:
↪")
    print(f"  {dropped}")
    print("\n  NOTE: on a temporal split, a family that lowers AUC is evidence_
↪it")
    print("  encodes period-specific behavior rather than durable signal._
↪Investigate")
    print("  before discarding permanently -- it may still help with a longer_
↪history")
    print("  or stricter leakage controls.")

```

Best feature set: + Timing / rhythm
 29 features, test AUC = 0.6835

Dropped 5 features that did not improve held-out AUC:
 ['prior_alert_count', 'prior_fp_count', 'prior_important_count',
 'days_since_last_alert', 'same_scenario_alert_count']

NOTE: on a temporal split, a family that lowers AUC is evidence it
 encodes period-specific behavior rather than durable signal. Investigate
 before discarding permanently -- it may still help with a longer history
 or stricter leakage controls.

1.7 5. Experiment 1 — Translation loss

Priority 1 in the plan. Compare the offline model’s *ideal* discard rate with the deployed SQL’s *actual* discard rate **on exactly the same residual population, at the same recall level.**

The comparison must be made at matched recall — comparing discard rates at different recall levels is meaningless.

Finding	Diagnosis	Action
Model discards materially more at same recall	Translation loss	Fix rule extraction (§6) — don’t add features
Both similar	Model itself is the ceiling	Feature/signal problem (§4) or accept and go to §8

```

[14]: def discard_rate_at_recall(proba, y_true, target_recall):
      """
      Lowest-scoring alerts are discarded first. Find the largest discard rate
      that still retains `target_recall` of important cases.
      """

```

```

y_true = np.asarray(y_true).astype(int)
order = np.argsort(proba)                                # ascending: safest-looking
↪first
important_sorted = y_true[order]
n_important = important_sorted.sum()
if n_important == 0:
    return 0.0, 0

cum_important_lost = np.cumsum(important_sorted)
recall_retained = 1 - cum_important_lost / n_important

ok = np.where(recall_retained >= target_recall)[0]
if len(ok) == 0:
    return 0.0, 0
k = ok[-1] + 1
return k / len(y_true), int(cum_important_lost[k - 1])

model_discard, model_lost = discard_rate_at_recall(
    best_model_result["proba"], test_df["is_important"], BASELINE_RECALL
)

print(f"At matched recall retained = {BASELINE_RECALL:.2%}:")
print(f"  Deployed SQL discard rate: {BASELINE_DISCARD:.2%}    (important lost:␣
↪{baseline['important_lost']})")
print(f"  Offline model capacity:    {model_discard:.2%}    (important lost:␣
↪{model_lost})")
print()

gap = model_discard - BASELINE_DISCARD
if gap > 0.03:
    print(f"  TRANSLATION LOSS = {gap:.2%}")
    print("  -> The model supports materially more discard than the SQL␣
↪extracts.")
    print("    PRIORITIZE SCORECARD TRANSLATION ($6), not more feature␣
↪engineering.")
elif gap > 0.0:
    print(f"  Modest translation gap = {gap:.2%} - some upside from better␣
↪extraction.")
else:
    print("  No translation loss - the model itself is the ceiling (feature/␣
↪signal problem).")

```

```

At matched recall retained = 91.57%:
  Deployed SQL discard rate:  17.54%    (important lost: 21)
  Offline model capacity:     21.82%    (important lost: 21)

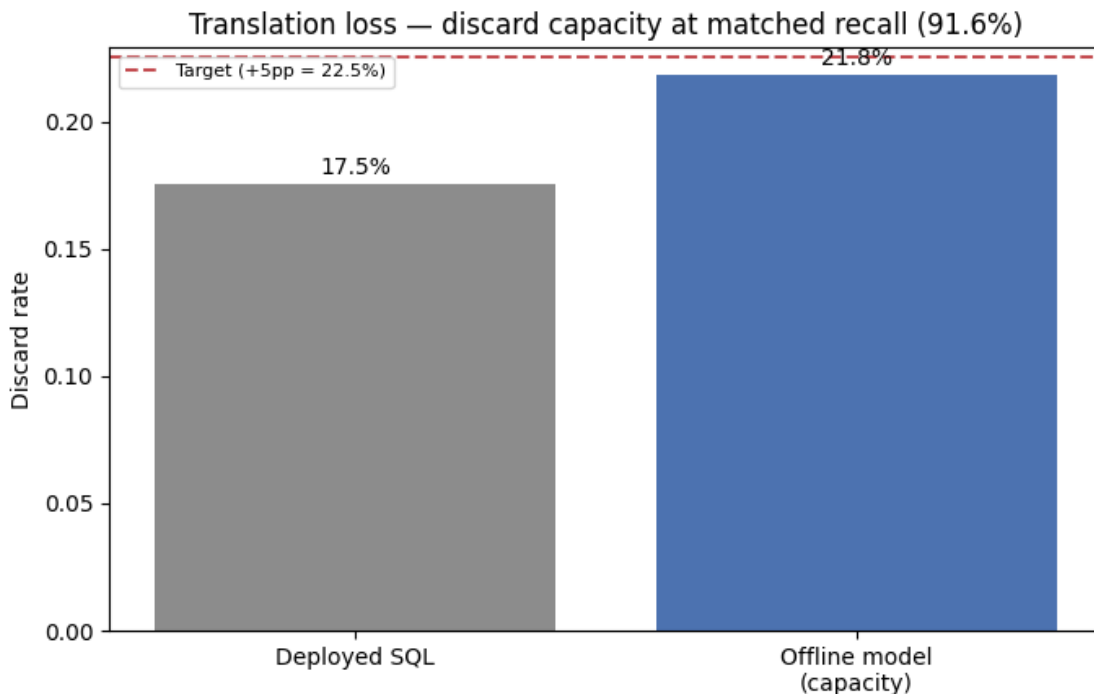
```

TRANSLATION LOSS = 4.28%

-> The model supports materially more discard than the SQL extracts.

PRIORITIZE SCORECARD TRANSLATION (\$6), not more feature engineering.

```
[15]: fig, ax = plt.subplots(figsize=(7, 4.5))
ax.bar(["Deployed SQL", "Offline model\n(capacity)"], [BASELINE_DISCARD,
↳model_discard],
      color=["#8c8c8c", "#4c72b0"])
ax.axhline(BASELINE_DISCARD + 0.05, color="#c44e52", linestyle="--",
          label=f"Target (+5pp = {BASELINE_DISCARD + 0.05:.1%})")
for i, v in enumerate([BASELINE_DISCARD, model_discard]):
    ax.text(i, v + 0.004, f"{v:.1%}", ha="center")
ax.set_ylabel("Discard rate")
ax.set_title(f"Translation loss - discard capacity at matched recall_
↳({BASELINE_RECALL:.1%})")
ax.legend(fontsize=8)
plt.tight_layout()
plt.show()
```



1.8 6. Experiment 2 — SHAP scorecard

1.8.1 6.1 Why AND-chained thresholds under-discard

This is the structural reason a Form-1 deployment plateaus.

An AND-chain requires an alert to clear **every** condition:

```
WHEN a < X AND b < Y AND c < Z AND d < W THEN 'DISCARD'
```

If each condition is individually satisfied by ~60% of alerts and they were independent, the joint discard rate is 0.60 **13%** — and correlation between conditions typically pushes it lower still. Each additional AND condition shrinks the discard population *multiplicatively*, no matter how well each threshold is individually chosen.

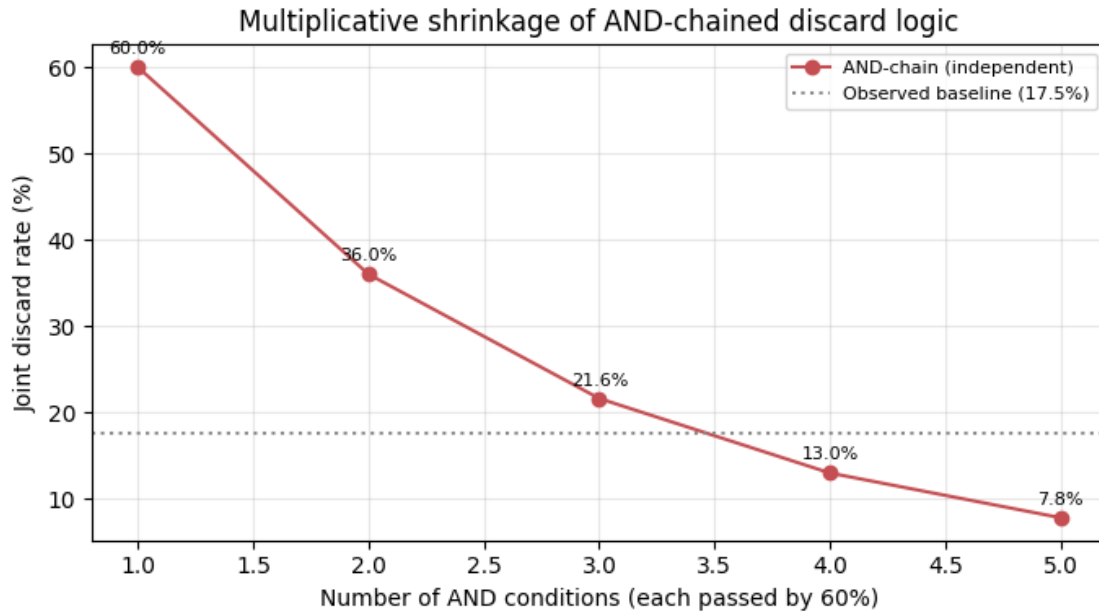
A **scorecard** changes the logic from conjunctive to additive:

```
WHEN (points_a + points_b + points_c + points_d) < T THEN 'DISCARD'
```

Now an alert doesn't need to be low on *everything* — just low *overall*. Moderate values across several features can still sum below the threshold. This is what recovers the interaction structure that independent thresholds discard.

```
[16]: # Illustration: AND-chain discard rate collapses as conditions are added
pass_rates = [0.60, 0.60, 0.60, 0.60, 0.60]
cumulative = np.cumprod(pass_rates)

fig, ax = plt.subplots(figsize=(7, 4))
ax.plot(range(1, len(cumulative) + 1), cumulative * 100, marker="o",
        color="#c44e52", label="AND-chain (independent)")
ax.axhline(BASELINE_DISCARD * 100, color="#8c8c8c", linestyle=":",
        label=f"Observed baseline ({BASELINE_DISCARD:.1%})")
for i, v in enumerate(cumulative):
    ax.text(i + 1, v * 100 + 1.5, f"{v:.1%}", ha="center", fontsize=8)
ax.set_xlabel("Number of AND conditions (each passed by 60%)")
ax.set_ylabel("Joint discard rate (%)")
ax.set_title("Multiplicative shrinkage of AND-chained discard logic")
ax.legend(fontsize=8)
ax.grid(alpha=0.3)
plt.tight_layout()
plt.show()
```



1.8.2 6.2 Build the scorecard from SHAP contributions

Per the plan (§3): use SHAP contributions to create **interpretable feature bins and points**, sum the points into a transparent risk score, calibrate the discard threshold on a temporally later unsampled population, and compile to SQL.

Because gradient-boosted trees are additive by construction and SHAP decomposes each prediction additively, a scorecard is a natural — not merely approximate — translation.

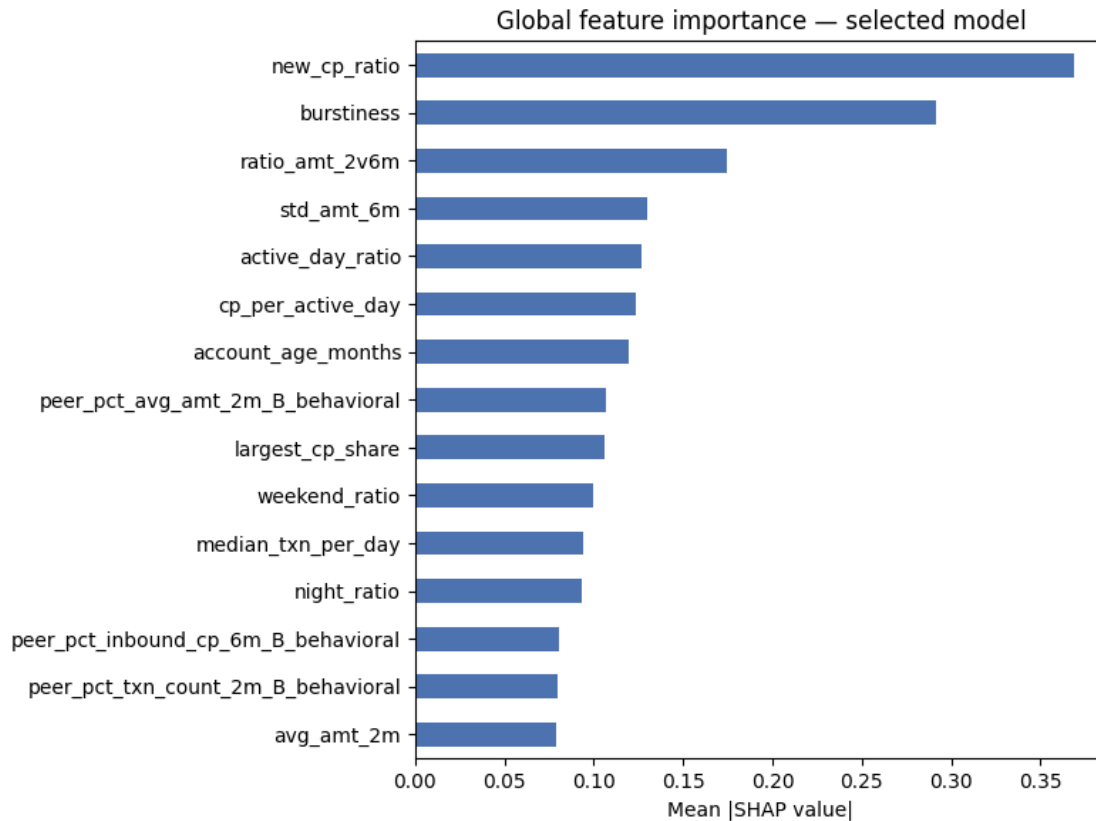
```
[17]: model = best_model_result["model"]
X_train_full = train_df[BEST_FEATURES]

explainer = shap.TreeExplainer(model)
shap_values = explainer.shap_values(X_train_full)
if isinstance(shap_values, list):
    shap_values = shap_values[1]

mean_abs = pd.Series(np.abs(shap_values).mean(axis=0), index=BEST_FEATURES).
    ↪sort_values(ascending=False)

fig, ax = plt.subplots(figsize=(8, 6))
mean_abs.head(15).iloc[::-1].plot(kind="barh", ax=ax, color="#4c72b0")
ax.set_xlabel("Mean |SHAP value|")
ax.set_title("Global feature importance - selected model")
plt.tight_layout()
plt.show()
```

```
SCORECARD_FEATURES = mean_abs.head(8).index.tolist()
print("Scorecard features (top 8 by mean |SHAP|):")
for f in SCORECARD_FEATURES:
    print(f" {f}")
```



Scorecard features (top 8 by mean |SHAP|):

```
new_cp_ratio
burstiness
ratio_amt_2v6m
std_amt_6m
active_day_ratio
cp_per_active_day
account_age_months
peer_pct_avg_amt_2m_B_behavioral
```

```
[18]: SCALE = 10.0 / max(np.abs(shap_values).max(), 1e-9)
```

```
def build_scorecard(features, shap_vals, X, n_bins=4):
    """Bin each feature; points = mean SHAP contribution in bin, rescaled to
    integers."""
```

```

cards = {}
for feat in features:
    col_idx = list(X.columns).index(feat)
    d = pd.DataFrame({"value": X[feat].values, "shap": shap_vals[:,
    ↪col_idx]}).dropna()
    try:
        d["bin"] = pd.qcut(d["value"], q=n_bins, duplicates="drop")
    except ValueError:
        d["bin"] = pd.cut(d["value"], bins=n_bins)
    tbl = (d.groupby("bin", observed=True)["shap"].mean() * SCALE).round().
    ↪astype(int)
    cards[feat] = tbl
return cards

scorecards = build_scorecard(SCORECARD_FEATURES, shap_values, X_train_full)

for feat, tbl in scorecards.items():
    print(f"\n--- {feat} ---")
    for interval, pts in tbl.items():
        print(f"    ({interval.left:>12.3f}, {interval.right:>12.3f}] -> {pts:
    ↪+d} points")

```

```

--- new_cp_ratio ---
(      -0.001,      0.250] -> -3 points
(      0.250,      0.400] -> +1 points
(      0.400,      1.000] -> +3 points

--- burstiness ---
(      -0.001,      0.215] -> -4 points
(      0.215,      0.392] -> -1 points
(      0.392,      0.574] -> +1 points
(      0.574,      1.463] -> +2 points

--- ratio_amt_2v6m ---
(      0.999,      1.038] -> -3 points
(      1.038,      1.072] -> -1 points
(      1.072,      1.123] -> +0 points
(      1.123,      1.725] -> +1 points

--- std_amt_6m ---
(      14.196,      474.713] -> -2 points
(      474.713,      1121.439] -> +0 points
(      1121.439,      2844.247] -> +0 points
(      2844.247,      178999.847] -> +0 points

```

```

--- active_day_ratio ---
(      0.010,      0.067] -> -1 points
(      0.067,      0.083] -> +0 points
(      0.083,      0.100] -> +0 points
(      0.100,      0.228] -> +1 points

--- cp_per_active_day ---
(     -0.001,      0.188] -> -1 points
(      0.188,      0.296] -> -1 points
(      0.296,      0.438] -> +0 points
(      0.438,      3.000] -> +1 points

--- account_age_months ---
(      0.999,     10.314] -> +0 points
(     10.314,     25.148] -> +0 points
(     25.148,     49.356] -> +0 points
(     49.356,    240.000] -> -2 points

--- peer_pct_avg_amt_2m_B_behavioral ---
(     -0.001,      0.251] -> +0 points
(      0.251,      0.501] -> +0 points
(      0.501,      0.751] -> +0 points
(      0.751,      1.000] -> -1 points

```

```

[19]: def apply_scorecard(df, scorecards):
        """Sum scorecard points across features. NaN contributes 0 (neutral)."""
        total = np.zeros(len(df))
        for feat, tbl in scorecards.items():
            vals = df[feat].values
            pts = np.zeros(len(df))
            for interval, p in tbl.items():
                mask = (vals > interval.left) & (vals <= interval.right)
                pts[mask] = p
            total += pts
        return total

test_scores = apply_scorecard(test_df, scorecards)
train_scores = apply_scorecard(train_df, scorecards)

fig, ax = plt.subplots(figsize=(8, 4.5))
ax.hist(test_scores[test_df["is_important"] == 0], bins=30, alpha=0.6,
        density=True,
        label="Not important", color="#4c72b0")
ax.hist(test_scores[test_df["is_important"] == 1], bins=30, alpha=0.6,
        density=True,
        label="Important", color="#c44e52")

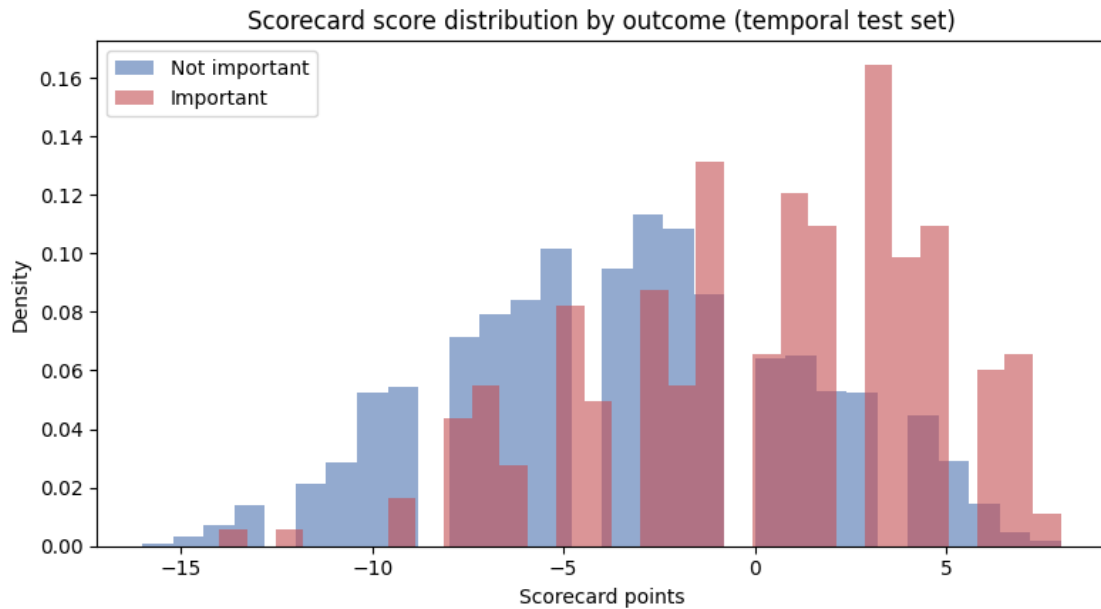
```

```

ax.set_xlabel("Scorecard points")
ax.set_ylabel("Density")
ax.set_title("Scorecard score distribution by outcome (temporal test set)")
ax.legend()
plt.tight_layout()
plt.show()

print(f"Scorecard AUC on test set: {roc_auc_score(test_df['is_important'],
↪test_scores):.4f}")
print(f"Selected model AUC on test set: {best_model_result['auc']:.4f}")

```



Scorecard AUC on test set: 0.7212
Selected model AUC on test set: 0.6835

1.8.3 6.3 Scorecard fidelity

Report this number in model documentation: *“the deployed scorecard recovers X% of the offline model’s discard capacity at the chosen operating point; the residual gap is accepted in exchange for full rule-engine deployability and auditability.”*

```

[20]: sc_discard, sc_lost = discard_rate_at_recall(test_scores,
↪test_df["is_important"], BASELINE_RECALL)

fidelity = sc_discard / model_discard * 100 if model_discard > 0 else np.nan

comparison = pd.DataFrame([

```

```

        {"approach": "Deployed SQL (AND-chain)", "discard_rate": BASELINE_DISCARD,
        ↪ "important_lost": baseline["important_lost"]},
        {"approach": "SHAP scorecard (additive)", "discard_rate": sc_discard,
        ↪ "important_lost": sc_lost},
        {"approach": "Selected model (reference)", "discard_rate": model_discard,
        ↪ "important_lost": model_lost},
    ])
    comparison["vs_baseline_pp"] = (comparison["discard_rate"] - BASELINE_DISCARD)
    ↪ * 100

    print(f"All measured at matched recall retained = {BASELINE_RECALL:.2%}\n")
    print(comparison.round(4).to_string(index=False))
    print(f"\nScorecard discard capacity vs. selected model: {fidelity:.1f}%")
    if fidelity > 100:
        print(" NOTE: the scorecard EXCEEDS the model here. This is not an error,
        ↪ -- binning")
        print(" the top features acts as regularization, and pruning to 8 features
        ↪ removes")
        print(" noise the fuller model was fitting. It is a real (and welcome)
        ↪ outcome on")
        print(" a temporal split, but verify it holds on a second time window
        ↪ before relying on it.")

    target_gain = 0.05
    achieved = sc_discard - BASELINE_DISCARD
    print(f"\nTarget gain: +{target_gain:.1%} Achieved by scorecard: {achieved:+.
    ↪ 2%}")
    print("TARGET MET" if achieved >= target_gain else "TARGET NOT YET MET -
    ↪ continue to §7/§8")

```

All measured at matched recall retained = 91.57%

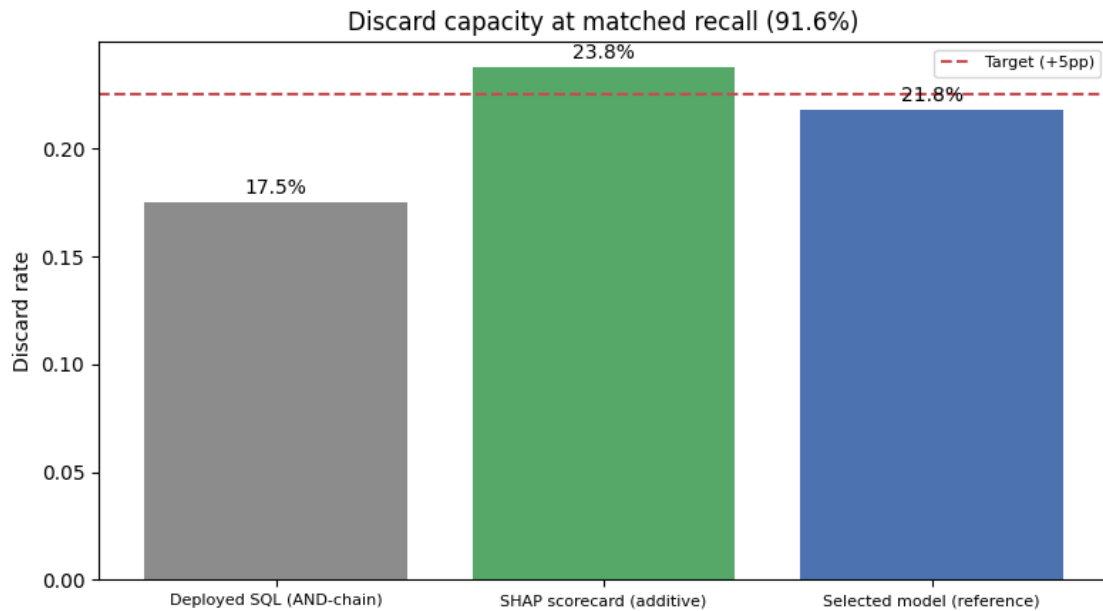
	approach	discard_rate	important_lost	vs_baseline_pp
	Deployed SQL (AND-chain)	0.1754	21	0.0000
	SHAP scorecard (additive)	0.2376	21	6.2257
	Selected model (reference)	0.2182	21	4.2802

Scorecard discard capacity vs. selected model: 108.9%

NOTE: the scorecard EXCEEDS the model here. This is not an error -- binning the top features acts as regularization, and pruning to 8 features removes noise the fuller model was fitting. It is a real (and welcome) outcome on a temporal split, but verify it holds on a second time window before relying on it.

Target gain: +5.0% Achieved by scorecard: +6.23%
TARGET MET

```
[21]: fig, ax = plt.subplots(figsize=(8, 4.5))
      colors = ["#8c8c8c", "#55a868", "#4c72b0"]
      bars = ax.bar(comparison["approach"], comparison["discard_rate"], color=colors)
      ax.axhline(BASELINE_DISCARD + 0.05, color="#c44e52", linestyle="--",
        ↪label="Target (+5pp)")
      for i, v in enumerate(comparison["discard_rate"]):
          ax.text(i, v + 0.004, f"{v:.1%}", ha="center")
      ax.set_ylabel("Discard rate")
      ax.set_title(f"Discard capacity at matched recall ({BASELINE_RECALL:.1%})")
      ax.legend(fontsize=8)
      plt.xticks(fontsize=8)
      plt.tight_layout()
      plt.show()
```



1.9 7. The discard-vs-recall frontier

Per the plan’s objective (§1): **do not optimize for a fixed 10% target first — establish the frontier.** This is the artifact to put in front of the MLRO, because it reframes the conversation from an abstract percentage to a concrete trade: “discarding $X\%$ costs us N important cases per period.”

```
[22]: def build_frontier(proba, y_true, label):
      y_true = np.asarray(y_true).astype(int)
      order = np.argsort(proba)
      imp_sorted = y_true[order]
      n_imp = imp_sorted.sum()
      cum_lost = np.cumsum(imp_sorted)
```

```

k = np.arange(1, len(y_true) + 1)
return pd.DataFrame({
    "discard_rate": k / len(y_true),
    "important_lost": cum_lost,
    "recall_retained": 1 - cum_lost / n_imp,
    "approach": label,
})

frontier_model = build_frontier(best_model_result["proba"],
    ↪test_df["is_important"], "Selected model")
frontier_score = build_frontier(test_scores, test_df["is_important"],
    ↪"Scorecard")

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

ax = axes[0]
for fr, c in [(frontier_model, "#4c72b0"), (frontier_score, "#55a868")]:
    ax.plot(fr["discard_rate"] * 100, fr["recall_retained"] * 100,
    ↪label=fr["approach"].iloc[0], color=c)
ax.scatter([BASELINE_DISCARD * 100], [BASELINE_RECALL * 100], color="#8c8c8c",
    ↪s=80,
            zorder=5, label="Current SQL baseline")
ax.axvline((BASELINE_DISCARD + 0.05) * 100, color="#c44e52", linestyle="--",
    ↪linewidth=1, label="Target (+5pp)")
ax.set_xlabel("Discard rate (%)")
ax.set_ylabel("Important-case recall retained (%)")
ax.set_title("Discard-vs-recall frontier")
ax.set_xlim(0, 50)
ax.set_ylim(70, 101)
ax.legend(fontsize=8)
ax.grid(alpha=0.3)

ax = axes[1]
for fr, c in [(frontier_model, "#4c72b0"), (frontier_score, "#55a868")]:
    ax.plot(fr["discard_rate"] * 100, fr["important_lost"],
    ↪label=fr["approach"].iloc[0], color=c)
ax.scatter([BASELINE_DISCARD * 100], [baseline["important_lost"]],
    ↪color="#8c8c8c", s=80,
            zorder=5, label="Current SQL baseline")
ax.axvline((BASELINE_DISCARD + 0.05) * 100, color="#c44e52", linestyle="--",
    ↪linewidth=1, label="Target (+5pp)")
ax.set_xlabel("Discard rate (%)")
ax.set_ylabel("Important cases lost (count, test period)")
ax.set_title("The MLRO conversation: what does each extra point of discard cost?
    ↪")

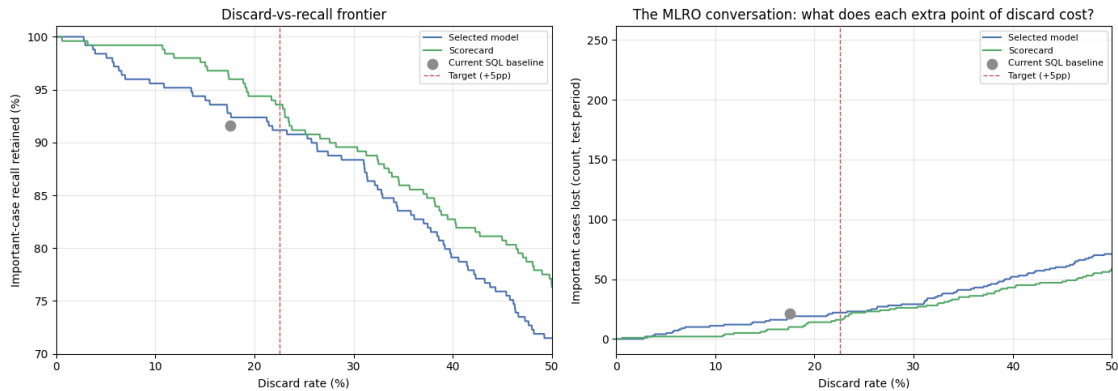
```

```

ax.set_xlim(0, 50)
ax.legend(fontsize=8)
ax.grid(alpha=0.3)

plt.tight_layout()
plt.show()

```



1.9.1 7.1 Loss-budget table

The plan’s decision sequence (§13) includes **“agree the permitted STR/important-case loss budget with the MLRO.”** This table makes that conversation concrete: for each candidate loss budget, what discard rate becomes available?

```

[23]: budget_rows = []
for budget in [0, 1, 2, 3, 5, 8, 12, 20]:
    for fr, name in [(frontier_model, "Full model"), (frontier_score,
↳ "Scorecard")]:
        ok = fr[fr["important_lost"] <= budget]
        if len(ok):
            row = ok.iloc[-1]
            budget_rows.append({
                "important_lost_budget": budget,
                "approach": name,
                "achievable_discard": row["discard_rate"],
                "recall_retained": row["recall_retained"],
                "gain_vs_baseline_pp": (row["discard_rate"] - BASELINE_DISCARD)
↳ * 100,
            })

budget_df = pd.DataFrame(budget_rows)
budget_pivot = budget_df.pivot(index="important_lost_budget",
↳ columns="approach",
                                values="achievable_discard")

```

```
print("Achievable discard rate by important-case loss budget (test period):\n")
print((budget_pivot * 100).round(1).to_string())
print(f"\nCurrent baseline: {BASELINE_DISCARD:.1%} discard, {BASELINE_IMPORTANT_LOST:.1%} important lost")
```

Achievable discard rate by important-case loss budget (test period):

approach	Full model	Scorecard
important_lost_budget		
0	2.8	0.6
1	2.9	3.2
2	3.7	10.7
3	3.9	11.0
5	5.7	14.5
8	6.6	17.3
12	13.6	19.2
20	21.4	23.5

Current baseline: 17.5% discard, 21 important lost

1.10 8. Experiment 8 — Three-tier operating design

The plan's recommended operating design (§12):

ALL ALERTS

```
|
+-- Mandatory / policy --> INVESTIGATE
|
+-- Eligible alerts --> TRIAGE SCORECARD
    |
    +-----+-----+
    |           |           |
    HIGH SCORE  MEDIUM SCORE  LOW SCORE
    |           |           |
    INVESTIGATE  LOW PRIORITY  DISCARD
```

Why this matters when the target isn't safely reachable. The underlying business need is investigator capacity, not literal deletion. A low-priority tier that is *sampled* rather than fully worked delivers most of the operational relief at far lower risk than permanent discard — and it keeps generating labels on the deprioritized population, which is exactly the evidence needed to justify true discard later.

```
[24]: DISCARD_CUT = np.percentile(test_scores, 20)      # bottom 20% -> discard
      LOWPRI_CUT = np.percentile(test_scores, 55)      # next 35% -> low priority

      tier = np.where(test_scores <= DISCARD_CUT, "DISCARD",
                      np.where(test_scores <= LOWPRI_CUT, "LOW_PRIORITY", "INVESTIGATE"))
```

```

tier_summary = (
    pd.DataFrame({"tier": tier, "is_important": test_df["is_important"].values})
    .groupby("tier")
    .agg(alerts=("is_important", "size"),
         important=("is_important", "sum"),
         important_rate=("is_important", "mean"))
    .reindex(["DISCARD", "LOW_PRIORITY", "INVESTIGATE"])
)
tier_summary["share_of_alerts"] = tier_summary["alerts"] /
    ↪ tier_summary["alerts"].sum()
tier_summary["share_of_important"] = tier_summary["important"] /
    ↪ tier_summary["important"].sum()

print(tier_summary.round(4).to_string())

SAMPLE_RATE = 0.20
lowpri = tier_summary.loc["LOW_PRIORITY"]
workload_removed = tier_summary.loc["DISCARD", "share_of_alerts"] +
    ↪ lowpri["share_of_alerts"] * (1 - SAMPLE_RATE)

print(f"\nWith a {SAMPLE_RATE:.0%} sampling rate on the low-priority tier:")
print(f" Effective workload reduction: {workload_removed:.1%}")
print(f" Permanently discarded:      {tier_summary.loc['DISCARD',
    ↪ 'share_of_alerts']:.1%}")
print(f" Important cases in discard:   {int(tier_summary.loc['DISCARD',
    ↪ 'important'])}")
print(f" Important cases in low-pri:   {int(lowpri['important'])} (recoverable,
    ↪ via sampling)")

```

	alerts	important	important_rate	share_of_alerts
share_of_important				
tier				
DISCARD	915	23	0.0251	0.2543
0.0924				
LOW_PRIORITY	1099	45	0.0409	0.3054
0.1807				
INVESTIGATE	1584	181	0.1143	0.4402
0.7269				

With a 20% sampling rate on the low-priority tier:

```

Effective workload reduction: 49.9%
Permanently discarded:      25.4%
Important cases in discard:  23
Important cases in low-pri:  45 (recoverable via sampling)

```

```
[25]: fig, axes = plt.subplots(1, 2, figsize=(13, 4.5))
```

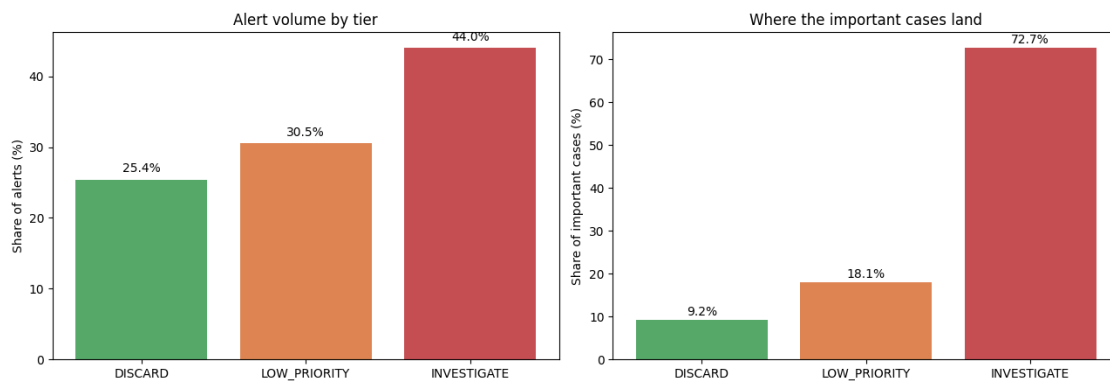
```

ax = axes[0]
ax.bar(tier_summary.index, tier_summary["share_of_alerts"] * 100,
      color=["#55a868", "#dd8452", "#c44e52"])
for i, v in enumerate(tier_summary["share_of_alerts"] * 100):
    ax.text(i, v + 1, f"{v:.1f}%", ha="center")
ax.set_ylabel("Share of alerts (%)")
ax.set_title("Alert volume by tier")

ax = axes[1]
ax.bar(tier_summary.index, tier_summary["share_of_important"] * 100,
      color=["#55a868", "#dd8452", "#c44e52"])
for i, v in enumerate(tier_summary["share_of_important"] * 100):
    ax.text(i, v + 1, f"{v:.1f}%", ha="center")
ax.set_ylabel("Share of important cases (%)")
ax.set_title("Where the important cases land")

plt.tight_layout()
plt.show()

```



1.10.1 8.1 Compile the scorecard to auditable SQL

```

[26]: def scorecard_to_sql(scorecards, discard_cut, lowpri_cut):
    terms = []
    for feat, tbl in scorecards.items():
        whens = []
        for interval, pts in tbl.items():
            lo = "-1e30" if np.isinf(interval.left) else f"{interval.left:.4f}"
            hi = "1e30" if np.isinf(interval.right) else f"{interval.right:.4f}"
            whens.append(f"    WHEN {feat} > {lo} AND {feat} <= {hi} THEN
        {pts}")
        terms.append("    CASE\n" + "\n".join(whens) + "\n    ELSE 0\n"
        "\nEND")

```

```

    score_expr = "\n    +\n".join(terms)
    return f"""WITH scored AS (
SELECT
    alert_id,
    customer_id,
{score_expr}
    AS risk_score
FROM residual_alert_features
)
SELECT
    alert_id,
    customer_id,
    risk_score,
CASE
    WHEN risk_score <= {discard_cut:.0f} THEN 'DISCARD'
    WHEN risk_score <= {lowpri_cut:.0f} THEN 'LOW_PRIORITY'
    ELSE 'INVESTIGATE'
END AS triage_decision
FROM scored;"""

print(scorecard_to_sql(scorecards, DISCARD_CUT, LOWPRI_CUT))

```

```

WITH scored AS (
SELECT
    alert_id,
    customer_id,
CASE
    WHEN new_cp_ratio > -0.0010 AND new_cp_ratio <= 0.2500 THEN -3
    WHEN new_cp_ratio > 0.2500 AND new_cp_ratio <= 0.4000 THEN 1
    WHEN new_cp_ratio > 0.4000 AND new_cp_ratio <= 1.0000 THEN 3
    ELSE 0
END
+
CASE
    WHEN burstiness > -0.0010 AND burstiness <= 0.2150 THEN -4
    WHEN burstiness > 0.2150 AND burstiness <= 0.3920 THEN -1
    WHEN burstiness > 0.3920 AND burstiness <= 0.5740 THEN 1
    WHEN burstiness > 0.5740 AND burstiness <= 1.4630 THEN 2
    ELSE 0
END
+
CASE
    WHEN ratio_amt_2v6m > 0.9990 AND ratio_amt_2v6m <= 1.0380 THEN -3
    WHEN ratio_amt_2v6m > 1.0380 AND ratio_amt_2v6m <= 1.0720 THEN -1
    WHEN ratio_amt_2v6m > 1.0720 AND ratio_amt_2v6m <= 1.1230 THEN 0
    WHEN ratio_amt_2v6m > 1.1230 AND ratio_amt_2v6m <= 1.7250 THEN 1
    ELSE 0

```

```

END
+
CASE
    WHEN std_amt_6m > 14.1960 AND std_amt_6m <= 474.7130 THEN -2
    WHEN std_amt_6m > 474.7130 AND std_amt_6m <= 1121.4390 THEN 0
    WHEN std_amt_6m > 1121.4390 AND std_amt_6m <= 2844.2470 THEN 0
    WHEN std_amt_6m > 2844.2470 AND std_amt_6m <= 178999.8470 THEN 0
    ELSE 0
END
+
CASE
    WHEN active_day_ratio > 0.0101 AND active_day_ratio <= 0.0667 THEN -1
    WHEN active_day_ratio > 0.0667 AND active_day_ratio <= 0.0833 THEN 0
    WHEN active_day_ratio > 0.0833 AND active_day_ratio <= 0.1000 THEN 0
    WHEN active_day_ratio > 0.1000 AND active_day_ratio <= 0.2280 THEN 1
    ELSE 0
END
+
CASE
    WHEN cp_per_active_day > -0.0010 AND cp_per_active_day <= 0.1880 THEN -1
    WHEN cp_per_active_day > 0.1880 AND cp_per_active_day <= 0.2960 THEN -1
    WHEN cp_per_active_day > 0.2960 AND cp_per_active_day <= 0.4380 THEN 0
    WHEN cp_per_active_day > 0.4380 AND cp_per_active_day <= 3.0000 THEN 1
    ELSE 0
END
+
CASE
    WHEN account_age_months > 0.9990 AND account_age_months <= 10.3140 THEN 0
    WHEN account_age_months > 10.3140 AND account_age_months <= 25.1480 THEN 0
    WHEN account_age_months > 25.1480 AND account_age_months <= 49.3560 THEN 0
    WHEN account_age_months > 49.3560 AND account_age_months <= 240.0000 THEN
-2
    ELSE 0
END
+
CASE
    WHEN peer_pct_avg_amt_2m_B_behavioral > -0.0006 AND
peer_pct_avg_amt_2m_B_behavioral <= 0.2510 THEN 0
    WHEN peer_pct_avg_amt_2m_B_behavioral > 0.2510 AND
peer_pct_avg_amt_2m_B_behavioral <= 0.5010 THEN 0
    WHEN peer_pct_avg_amt_2m_B_behavioral > 0.5010 AND
peer_pct_avg_amt_2m_B_behavioral <= 0.7510 THEN 0
    WHEN peer_pct_avg_amt_2m_B_behavioral > 0.7510 AND
peer_pct_avg_amt_2m_B_behavioral <= 1.0000 THEN -1
    ELSE 0
END
AS risk_score
FROM residual_alert_features

```

```

)
SELECT
  alert_id,
  customer_id,
  risk_score,
  CASE
    WHEN risk_score <= -7 THEN 'DISCARD'
    WHEN risk_score <= -3 THEN 'LOW_PRIORITY'
    ELSE 'INVESTIGATE'
  END AS triage_decision
FROM scored;

```

1.11 9. Decision sequence and results

1.11.1 9.1 Where this run landed

```

[27]: summary = pd.DataFrame([
    {"step": "1. Freeze baseline", "result": f"{BASELINE_DISCARD:.1%} discard, {baseline['important_lost']} important lost"},
    {"step": "2. Residual AUC (baseline feats)", "result": f"AUC = {baseline_result['auc']:.4f}"},
    {"step": "3. + Counterparty breadth", "result": f"AUC = {cp_result['auc']:.4f} ({cp_result['auc'] - baseline_result['auc']:+.4f})"},
    {"step": f"4. + Peer deviation ({best_peer_def})", "result": f"AUC = {peer_results[best_peer_def]['auc']:.4f} ({peer_results[best_peer_def]['auc'] - cp_result['auc']:+.4f})"},
    {"step": "5. + Timing / rhythm", "result": f"AUC = {timing_result['auc']:.4f} ({timing_result['auc'] - peer_results[best_peer_def]['auc']:+.4f})"},
    {"step": "6. + Cross-alert history", "result": f"AUC = {full_result['auc']:.4f} ({full_result['auc'] - timing_result['auc']:+.4f})"},
    {"step": "7. Translation loss", "result": f"model {model_discard:.1%} vs SQL {BASELINE_DISCARD:.1%} = {gap:+.1%}"},
    {"step": "8. Scorecard discard", "result": f"{sc_discard:.1%} ({achieved:+.1%} vs baseline), fidelity {fidelity:.0f}%"},
    {"step": "9. Three-tier workload", "result": f"{workload_removed:.1%} effective reduction at {SAMPLE_RATE:.0%} sampling"},
])
summary

```

[27]:	step	result
0	1. Freeze baseline	17.5% discard, 21 important lost
1	2. Residual AUC (baseline feats)	AUC = 0.6003
2	3. + Counterparty breadth	AUC = 0.6557 (+0.0554)
3	4. + Peer deviation (B_behavioral)	AUC = 0.6682 (+0.0125)
4	5. + Timing / rhythm	AUC = 0.6835 (+0.0152)

5	6. + Cross-alert history	AUC = 0.6655 (-0.0179)
6	7. Translation loss	model 21.8% vs SQL 17.5% = +4.3%
7	8. Scorecard discard	23.8% (+6.2% vs baseline), fidelity 109%
8	9. Three-tier workload	49.9% effective reduction at 20% sampling

1.11.2 9.2 Decision sequence (plan §13)

1. ~~Freeze the current baseline~~ → §2
2. ~~Measure offline model discard capacity at the same recall~~ → §5
3. ~~If translation loss exists, fix scorecard/rule extraction~~ → §6
4. ~~Measure residual AUC~~ → §3
5. ~~If signal is weak, add counterparty breadth and peer deviation before more ratios~~ → §4
6. ~~Produce the discard vs recall frontier~~ → §7
7. **Agree the permitted important-case loss budget with the MLRO** → §7.1 table
8. **If the target is not safely achievable, use a low-priority tier** → §8

Steps 7-8 are decisions, not computations — the notebook produces the evidence, the MLRO chooses the operating point.

1.11.3 9.3 Sprint plan

Sprint	Scope	Notebook sections
1	Freeze baseline, construct residual population, residual AUC, offline-vs-SQL discard capacity	§1, §2, §3, §5
2	Scorecard, counterparty breadth, scenario/behavioral/hybrid peer groups	§4.1, §4.2, §6
3	Timing/rhythm, leakage-safe cross-alert history, repeat temporal validation	§4.3, §4.4
4	Select operating threshold with MLRO, compile auditable SQL, consider low-priority tier	§7, §8

1.11.4 9.4 Notes for the real-data run

- **Replace both stubs:** `load_residual_population()` (§1.2) and `current_sql_discard_rule()` (§2) — the latter must transcribe deployed logic exactly, or the translation-loss measurement is meaningless.
- **Peer-group reference distributions** are computed on training rows only (§4.2). Keep that discipline; recomputing on the full period leaks future information backward.
- **Cross-alert history** must use only alerts strictly before the current alert date, and must never encode the current case's own outcome.
- **Calibrate the final cut on an unsampled, temporally later slice** — thresholds chosen on resampled data do not transfer to production's true imbalance.

- **Prefer class weighting over SMOTE** — synthetic minority rows corrupt the SHAP contributions that the scorecard points are derived from.
- If AUC is materially lower on real data than here, that is the expected outcome of the residual population being genuinely harder than synthetic data — the frontier and three-tier design (§7-8) remain the useful deliverables regardless.